Vakint reduction benchmark

Cached RustRed versus the FORM / FMFT reduction route

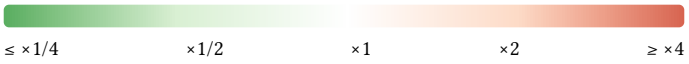
2026-10-20 · AMD EPYC 9754 128-Core Processor · CPU affinity 32

Reading guide. Bold values are RustRed times; FMFT times appear below. Multipliers are *RustRed ÷ FMFT*: green means faster, white parity, red slower. Both receive identical four-loop scalar inputs. The first physical denominator has power 2 (base) or 4 (held out); all others have power 1, auxiliaries 0.

Reduction time

Median seconds over 3 fresh processes per backend and case.

Topology	Base · power 2	Held out · power 4
H	**0.265 s** FMFT 0.852 s ×0.31	**3.54 s** FMFT 7.35 s ×0.48
FG	**0.0199 s** FMFT 0.157 s ×0.13	**0.0801 s** FMFT 0.338 s ×0.24
BMW	**0.0124 s** FMFT 0.118 s ×0.11	**0.796 s** FMFT 3.03 s ×0.26
X	**0.116 s** FMFT 1.55 s ×0.075	**13.5 s** FMFT 20.8 s ×0.65



Method and correctness. Release build: optimization level 3, fat LTO; rustc 1.98.1; FORM 4.3. Both retain masters in FMFT/MATAD normalization, with identical symbolic mass. Exact rational parity passed all eight cases at unit mass and renormalization scale, without truncating epsilon or using numerical tolerances.

Base cases hit prepared roots without applying rules. Held-out cases reuse cached descendants after root misses. Timing covers first public reduction calls, including FORM version checks, launch, I/O and parsing. Data loading, cache attachment, master evaluation and validation are excluded. Fresh processes run sequentially, alternating backends, with the recorded CPU affinity on a shared host. Filesystem pages are warm; this does not measure cold startup.

Historical measurements, unchanged. Source: `143122fb0934`. Data: `v0.1.0`. Samples and provenance: `data/results.json`. Refresh instructions: `../../AGENTS.md`.

Five-loop DOT ingress acceptance

640 unevaluated masters · debug verification

2026-10-23 · AMD EPYC 9754 128-Core Processor

Measurement boundary. This is one sequential verification pass through all published five-loop master graphs, using the native FeynKit parser and Vakint input conversion. It is a debug build with exact assertions enabled. It does not time package loading, IBP reduction, master evaluation or cache preparation. The release/FMFT comparison on page 1 retains its original date, source revision and measurements.

Validation workload	Result
Master graphs checked	640 / 640
Graphs with dotted propagators	441
Graphs with numerator factors	204
Exact routing and complete numerator checks	Passed
Native DOT round trips and Vakint ingress	Passed
Five-loop numerical values / prepared cache	Absent / absent

61.8 s complete graph-validation pass

Per graph: 0.0443–0.175 s; median 0.0955 s. One observed run; no statistical performance claim.

What was checked. Each stored family and signed index tuple is mapped exactly to the graph's physical propagators, powers, mass terms and complete numerator. Loop-basis annotations, including slot zero, and the integration-variable Jacobian are verified. The same integral is then reconstructed through native Vakint ingress and a FeynKit DOT round trip. These checks establish the graph definitions; they do not claim linear independence of the 640 retained masters.

Implementation relevant to this pass. Vakint now substitutes contracted scalar products directly inside factored numerators. FeynKit accepts identical stored numerator expressions before attempting expanded comparison. Both changes avoid unnecessary expansion while retaining exact fallback checks. Earlier interrupted checks are not a completed timing baseline, so no speedup factor is inferred from them.

Consumer base: `68f2263bf54d` (modified source; exact patch and executable digests recorded in JSON). RustRed: `52c7463686b4` Symbolica: `b6cca776f9dc`. Build: dev, unoptimized, debuginfo 2; rustc 1.97.1. Measured receipt and data manifest digests: `data/results.json → five_loop_dot_ingress_acceptance`.